Chapter 37 - Keyboard, Mouse, Controllers, and Time of Day

Input arrives through the terminal register block at `$F0700-$F07FF`. BASIC usually reads characters with `GET` and `INPUT`; games, editors, and music tools can also poll the raw registers directly. Controller state has its own shared block at `$F25C0-$F25FF`.

All registers in this chapter are 32-bit on the bus. Most values use only the low byte. Mouse X and Y use the low 16 bits. Relative mouse movement uses signed 32-bit values.

37.1 Register Map

Address	Name	R/W	Meaning
`$F0728`	`TERM_KEY_IN`	R	Read one raw key byte and advance the key queue.
`$F072C`	`TERM_KEY_STATUS`	R	Bit `0` set when a raw key byte is queued.
`$F0740`	`SCAN_CODE`	R	Read one physical-key scancode and advance the scancode queue.
`$F0744`	`SCAN_STATUS`	R	Bit `0` set when a scancode is queued.
`$F0748`	`SCAN_MODIFIERS`	R	Bit `0` shift, bit `1` ctrl, bit `2` alt, bit `3` capslock.
`$F0730`	`MOUSE_X`	R	Absolute mouse X, low `16` bits.
`$F0734`	`MOUSE_Y`	R	Absolute mouse Y, low `16` bits.
`$F0738`	`MOUSE_BUTTONS`	R	Bit `0` left, bit `1` right, bit `2` middle.
`$F073C`	`MOUSE_STATUS`	R	Bit `0` set after a mouse change; reading clears it.
`$F074C`	`MOUSE_CTRL`	R/W	Bit `0` requests relative captured mouse mode.
`$F0750`	`RTC_EPOCH`	R	Seconds since `1970-01-01 00:00:00` UTC.
`$F0754`	`MOUSE_DX`	R	Signed accumulated relative X movement; reading clears it.
`$F0758`	`MOUSE_DY`	R	Signed accumulated relative Y movement; reading clears it.
`$F075C`	`RTC_MONO_USEC_LO`	R	Low `32` bits of monotonic microseconds since engine start.
`$F0760`	`RTC_MONO_USEC_HI`	R	High `32` bits of monotonic microseconds since engine start.

37.2 Raw Key Queue

The raw key queue contains one byte per terminal key after the graphical input path has delivered it to the MMIO block. It is not line-buffered and it is not echoed. Use it when a program wants immediate key presses but still wants the usual character values.

```
10 REM WAIT FOR ONE RAW KEY BYTE
20 PRINT "PRESS A KEY"
30 IF PEEK32(&H000F072C)=0 THEN GOTO 30
40 K=PEEK32(&H000F0728)
50 PRINT "KEY ";K
```

If you press `A`, the final line prints `KEY 65`. Reading `$F0728` when the queue is empty returns `0`, so check `$F072C` first when zero is a meaningful key value for your program. Line `30` is the guard. Line `40` consumes the byte, so reading it again would move on to the next queued key.

37.3 Physical Scancodes

The scancode queue reports physical key events. A press and release both appear in the queue. The high bit marks release, so `30` and `158` are the press and release forms of the same key code.

```
10 REM SHOW PRESS AND RELEASE SCANCODES
20 PRINT "PRESS AND RELEASE A KEY"
30 IF PEEK32(&H000F0744)=0 THEN GOTO 30
40 C=PEEK32(&H000F0740)
50 M=PEEK32(&H000F0748)
60 PRINT "SCAN ";C;" MOD ";M
70 IF (C AND 128)=0 THEN GOTO 30
```

The modifier byte is not queued. It reports the modifier state at the moment it is read. Line `70` keeps the loop alive until the release form of the scancode appears. That release form is the press code plus `128`.

37.4 Absolute Mouse

Absolute mouse mode reports the current pointer position and button state. `MOUSE_STATUS` is a one-shot changed bit: it reads `1` after a change and clears to `0` when read.

```
10 REM READ STATUS FIRST, BECAUSE IT CLEARS ON READ
20 S=PEEK32(&H000F073C)
30 X=PEEK32(&H000F0730)
40 Y=PEEK32(&H000F0734)
50 B=PEEK32(&H000F0738)
60 PRINT "MOUSE ";X;Y;B;" CHANGED ";S
```

The button value is a bit field. Left is `1`, right is `2`, middle is `4`, and combined buttons add those values. Read `$F073C` once per sample. A second read before the next mouse event returns `0`.

37.5 Relative Mouse

Relative mode is for games and editors that care about movement rather than pointer position. Write `1` to `$F074C` to request captured relative mode. Write `0` to return to normal absolute mode.

```
10 REM CAPTURE RELATIVE MOVEMENT UNTIL A KEY IS PRESSED
20 POKE32 &H000F074C,1
30 PRINT "MOVE MOUSE, THEN PRESS A KEY"
40 IF PEEK32(&H000F072C)=0 THEN GOTO 40
50 K=PEEK32(&H000F0728)
60 DX=PEEK32(&H000F0754):DY=PEEK32(&H000F0758)
70 PRINT "DELTA ";DX;DY;" KEY ";K
80 POKE32 &H000F074C,0
```

`MOUSE_DX` and `MOUSE_DY` clear independently when read. Poll once per frame if you want frame-by-frame movement. Negative movement is reported as a signed 32-bit value. If BASIC prints a value greater than `2147483647`, subtract `4294967296` to view it as a negative number.

37.6 Time Of Day

`RTC_EPOCH` reads whole seconds since `1970-01-01 00:00:00` UTC. This is wall-clock time. It can jump if the clock is changed outside the machine, so use it for dates, not for measuring short intervals.

```
10 T=PEEK32(&H000F0750)
20 PRINT "SECONDS ";T
```

Two reads about a second apart normally differ by one. For shorter elapsed-time measurements, use the monotonic microsecond registers in the next section, a CPU timer, or a device status bit.

The value is a signed 32-bit seconds counter. In 2038 it crosses from positive to negative, then keeps counting in signed arithmetic.

37.7 Monotonic Elapsed Time

`RTC_MONO_USEC_LO` and `RTC_MONO_USEC_HI` form a 64-bit microsecond counter since Intuition Engine started. It is monotonic: it is for intervals and timeouts, not for calendar time.

Read the high word, then the low word, then the high word again. If the two high reads differ, the low word crossed `$FFFFFFFF` while you were reading and you should try again.

```
10 REM READ MONOTONIC MICROSECOND TIMER
20 H1=PEEK32(&H000F0760)
30 L=PEEK32(&H000F075C)
40 H2=PEEK32(&H000F0760)
50 IF H1<>H2 THEN GOTO 20
60 PRINT "USEC ";H2;L
```

On a short run the high word is usually `0`, and the low word is the elapsed microsecond count. For long-running programs, keep the value as two words. Even though BASIC uses double-precision numbers, not every 64-bit integer can be represented as one exact decimal value.

37.8 Gamepad Input

The input system samples up to four gamepads once per displayed frame and publishes one vendor-neutral layout for every CPU. The block at `$F25C0-$F25FF` is read-only from the programme's point of view.

Address	Name	R/W	Meaning
$F25C0	GAMEPAD_STATUS	R	Bits 0..3 pad connected mask, bits 8..11 connected count.

Addresses $F25C4 through $F25CF are reserved and read as zero.

Each pad p (0..3) has a 12-byte record at $F25D0 + p*$0C:

Offset	Name	R/W	Meaning
+$00	BUTTONS	R	Canonical button bitfield for this frame.
+$04	AXIS_LXY	R	Left stick, X in low 16 bits, Y in high 16 bits (signed).
+$08	AXIS_RXY	R	Right stick, X in low 16 bits, Y in high 16 bits (signed).

Bit	Mask	Button	Bit	Mask	Button
0	$00000001	Up	1	$00000002	Down
2	$00000004	Left	3	$00000008	Right
4	$00000010	A	5	$00000020	B
6	$00000040	X	7	$00000080	Y
8	$00000100	LB	9	$00000200	RB
10	$00000400	LT	11	$00000800	RT
12	$00001000	Select	13	$00002000	Start
14	$00004000	L3	15	$00008000	R3
16	$00010000	Home			

LT and RT are digital buttons. Each stick axis ranges from -32767 to 32767. Left and up are negative; right and down are positive. A disconnected slot reads as zero, including all previous button and axis state. Writes are accepted but ignored, and reads have no side effects.

BASIC provides PAD(n) for the button bitfield and PADX(n) and PADY(n) for the signed left-stick axes. The index is a slot from 0 through 3; a disconnected or out-of-range slot returns 0.

37.8.1 A moving and sounding controller test

This programme uses the left stick to move a VGA pixel and the A button to open a SoundChip gate:

```
10 REM GAMEPAD CURSOR AND BUTTON TONE
20 SCREEN 13
30 POKE32 &H000F0800,1
40 ENVELOPE 0,5,40,160,30
50 AM=16:OLD=0
55 FOR F=1 TO 600
60 X=160+INT(PADX(0)/512)
70 Y=100+INT(PADY(0)/512)
80 CLS
90 PLOT X,Y,15
100 B=PAD(0):A=B AND AM
110 IF A=0 THEN GATE 0,OFF
120 IF A<>0 AND OLD=0 THEN SOUND 0,660,180,0,128:GATE 0,ON
130 OLD=A
140 NEXT F
150 GATE 0,OFF
```

Lines `20` and `80` prepare a Mode 13h VGA frame. Lines `60` and `70` scale the signed left-stick axes into a safe area around the centre of the screen, and line `90` plots the current position. Line `50` defines the A-button mask locally, so the listing needs no library file. Lines `100` through `130` detect the button edge: pressing A starts the tone, and releasing A closes the gate. Line `55` keeps the demonstration running for `600` samples. Increase that value for a longer test. Line `150` leaves the SoundChip gate closed when the programme ends.

Expected result: while the programme runs, the bright pixel follows the left stick and a tone sounds while A is held. With no gamepad in slot `0`, the pixel remains at the centre and the sound stays off.

The raw block can also be inspected directly:

```
PRINT HEX$(PEEK32(&H000F25C0))
PRINT HEX$(PEEK32(&H000F25D0))
```

The first value contains the connection mask and count. The second is pad `0`'s button word. The exact values depend on the connected pads and buttons held during that frame.

37.9 Small-CPU Access

The 6502 and Z80 reach the terminal registers through the terminal bank described in Chapters 27 and 28. A `32`-bit register appears as four little-endian byte lanes. For most terminal registers the useful bits are in the low byte. For `MOUSE_X` and `MOUSE_Y`, read the low two bytes for the `16`-bit coordinate.

The gamepad block uses Bank 1 value `$79`, not the terminal bank. Use `SET_GAMEPAD_BANK`, then read `GAMEPAD_STATUS` at `$25C0` and pad `0` at `$25D0`. Button bits `0` through `7` are in `$25D0`, bits `8` through `15` in `$25D1`, and Home at bit `0` of `$25D2`. Signed axis halves use the same little-endian byte order. Selecting another bank later replaces this view of `$2000-$3FFF`.

The cooked-key register at `$F0728` shares its queue with terminal character input. A program should choose one consumer for that queue: BASIC `GET`, BASIC `INPUT`, terminal reads, or direct reads from `$F0728`.